

Istruzioni. Durante lo svolgimento del test, attenersi rigorosamente alle seguenti istruzioni.

- Gli esercizi devono essere consegnati attraverso il sito Moodle, al seguente indirizzo:

<https://elearning.unica.it/course/view.php?id=188>

- Ogni esercizio deve essere consegnato su un file con estensione `.ml`. Non sono ammesse altre modalità di consegna.
- Non è possibile consegnare alcun esercizio dopo la scadenza stabilita. È possibile re-inviare ogni esercizio un numero arbitrario di volte entro la scadenza stabilita, senza subire penalizzazioni.
- Non è ammesso alcun tipo di collaborazione.
- Se viene richiesto che una funzione abbia un nome specifico, la funzione deve avere quel nome.
- Il file contenente la soluzione non deve contenere esempi (anche commentati).
- Nelle soluzioni degli esercizi 7 e 8 il tipo deve essere contenuto nel file.

1. Scrivere una funzione `foo` con il seguente tipo:

`foo: ('a -> int) -> 'a -> int -> 'a`

2. Si consideri la successione $(a_n, b_n)_{n \in \mathbb{N}}$ di coppie di numeri interi data da:

$$\begin{aligned} a_0 &= 0 & b_0 &= 1 \\ a_{n+1} &= \begin{cases} 2b_n + 1 & \text{se } b_n \text{ pari} \\ 2b_n & \text{altrimenti} \end{cases} & b_{n+1} &= \begin{cases} a_n + 1 & \text{se } n \text{ dispari} \\ a_{n+1} & \text{altrimenti} \end{cases} \end{aligned}$$

Scrivere una funzione `seq: int -> (int * int)` tale che l'espressione `seq n` valuta alla coppia (a_n, b_n) . Nota: è consentito definire funzioni ausiliarie.

3. Date due funzioni $f, g: \mathbb{Z} \rightarrow \mathbb{Z}$, si definisca la funzione $f \circ g: \mathbb{Z} \rightarrow \mathbb{Z}$ come segue:

$$(f \circ g)(x) = \begin{cases} |g(x) - f(x)| & \text{se } g(x) \text{ è pari} \\ 2 * f(x) & \text{altrimenti} \end{cases}$$

Definire una funzione `comp` con il seguente tipo:

`comp: (int -> int) -> (int -> int) -> (int -> int)`

e tale che `comp f g = f ∘ g`.

4. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> 'b list -> ('b -> 'a) -> 'a list
```

5. Definire una funzione `fool` con tipo:

```
'a list -> int -> ('a -> bool) -> bool
```

in modo che `fool l n f` valuti a `true` se almeno uno dei primi `n` elementi di `l` soddisfa il predicato `p`, false altrimenti. Ad esempio,

```
fool [10;11;12;13;14;15;16;17;18;19;20] 7 (fun x -> x>15) = true
```

```
fool [10;11;12;13;14;15;16;17;18;19;20] 3 (fun x -> x>15) = false
```

6. Scrivere una funzione `foo` con il seguente tipo:

```
foo: ('a -> 'b) -> 'a list -> 'b list * 'b list
```

7. Si consideri il seguente tipo:

```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

Definire una funzione `sumEven` con tipo:

```
'int tree -> int
```

in modo che `sumEven t` restituisca la somma dei valori dei nodi di profondità pari di `t` (si assume che la radice sia al livello 0). Ad esempio:

```
# let t1 = Node (1, Node(2, Node( 3, Empty, Empty), Node(4, Empty, Empty)),
```

```
Node ( 5, Empty, Empty));;
```

```
# sumEven t1;;
```

```
- : int = 8
```

8. Si consideri la seguente definizione di tipo:

```
type 'a exp = Pip
```

```
| Plu of int * 'a exp
```

```
| Pap of 'a exp * 'a exp ;;
```

Definire una funzione `eval` che assegni ad un'espressione di tipo `exp` un valore di tipo `int`, interpretando `Pip` con 0, `Plu of int * 'a exp` con il valore dell'espressione sommato con l'intero, e `Pap of 'a exp * 'a exp` con il prodotto dei valori delle espressioni.